Script-Controlled ACL

Customer / User Journey Map — Restrict Record Access Based on Field Value

SCENARIO	1 SETUP	2 BUILD	3 SECURE	4 REQUEST	5 OPERATE	6 VERIFY
Configure and test role- and field-controlled record access.	Create the test identity and access roles	Create the table, fields and records	Configure the Read ACL and field condition	User requests record access or an operation	Create / Write / Delete according to roles	Check each user state against expected access
STEPS What happens in this journey?	Create EEE User Create bb1, bb2, bb3, bb4 Assign required roles	Create Institution Details table Use name u_institution_details Add required fields	Create ECE / EEE / CSE records Configure active advanced Read ACL Set Branch is EEE + supplied script	Impersonate EEE User Open Student Records list Request a record operation	bb1 → Read bb2 → Create bb3 → Write bb4 → Delete	Check user without bb1 Check EEE User Check Administrator
INTERACTIONS What does the person interact with?	ServiceNow User Administration Users / Roles records Role assignment	System Definition → Tables Table fields Table records / Branch value	System Security → Access Control Read ACL form Role, condition and script	Student Records list Current user / session ServiceNow access result	Create ACL Write ACL Delete ACL	Impersonation Record visibility New / Edit / Delete availability
GOALS & MOTIVATIONS What does the person want to achieve?	Create the exact test user Prepare the four required roles Make the role setup ready for ACL testing	Build the exact data structure Store student details Create different Branch values for testing	Restrict Read access Allow the intended EEE records Keep administrator access	Know whether access is allowed See the actual ACL result Compare different user states	Control Create, Write and Delete Apply the required role at each operation Progressively extend permissions	Confirm the expected outcome Verify all required role combinations Produce evidence for the project
POSITIVE MOMENTS What works well?	User and roles are created correctly.	Fields and Branch choices are available.	Read ACL matches the supplied configuration.	Expected EEE/Admin visibility is observed.	CRUD access changes with bb2, bb3 and bb4.	Verification results match the resource.
NEGATIVE MOMENTS What causes friction?	Wrong role assignment affects testing.	Missing fields or records affect testing.	Wrong ACL role, condition or script changes access.	Unauthorized user may receive an unexpected result.	Missing role prevents the intended operation.	Incomplete verification leaves the result unproven.
AREAS OF OPPORTUNITY How can the journey be improved?	Verify role assignments before testing.	Use the exact fields and Branch choices.	Keep the supplied Read ACL logic unchanged.	Test with the specified user states.	Test each CRUD ACL separately.	Capture evidence for each verification case.

ACCESS LOGIC

Read: bb1 + Branch = EEE + supplied script | Create: bb2 | Write: bb3 | Delete: bb4 | Administrator: full Read access